

Practical 4

- Two programs has been created using gedit.
- The child processes displayed their process IDs, while the parent synchronized their completion using wait() and waitpid().
- A separate zombie-process program was also executed, where the parent waited for the child before completing.

Multiple Child Processes Output

```
vida@LAPTOP-4B7UUAUH:~/os_lab$ gedit sleepexit1.c
** (gedit:3302): WARNING **: 17:14:45.133: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:3302): WARNING **: 17:14:45.133: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
vida@LAPTOP-4B7UUAUH:~/os_lab$ gcc sleepexit1.c
vida@LAPTOP-4B7UUAUH:~/os_lab$ ./a.out
child 1 pid = 3678
child 2 pid = 3679
child 3 pid = 3680
parent:Any one child complete with wait()
parent: child 3 completes with waitpid()
all the childs completed
vida@LAPTOP-4B7UUAUH:~/os_lab$ |
```

Three child processes are created with PIDs 3678, 3679 and 3680. The parent responded completion using wait()

Then completion of child 3 using waitpid().

Then it confirmed that all children processes are completed.

Zombie Process Output

```
vida@LAPTOP-4B7UUAUH:~/os_lab$ gcc zombie.c
vida@LAPTOP-4B7UUAUH:~/os_lab$ ./a.out
parent process 2886
child process 2887
Parent process completedvida@LAPTOP-4B7UUAUH:~/os_lab$ |
```

- The parent process (PID 2886) and child process (PID 2887) were created successfully.
- The child completed, and the parent displayed "Parent process completed"
- A zombie process occurs when a child process terminates but the parent has not yet collected its exit status. The process table can be investigated using the ps command. Using wait() or waitpid() allows the parent to collect the child's termination status and removes the zombie entry.